An Efficiency-Aware Convolutional Neural Network for Medical Image Classification

Group 11
Hung Siu Wang 57148331
Chan Tsz Hin 58539653
Matteo Zabban 40172615
Choi San Ho 57849436

INTRODUCTION

Why Skin-Lesions Classification?

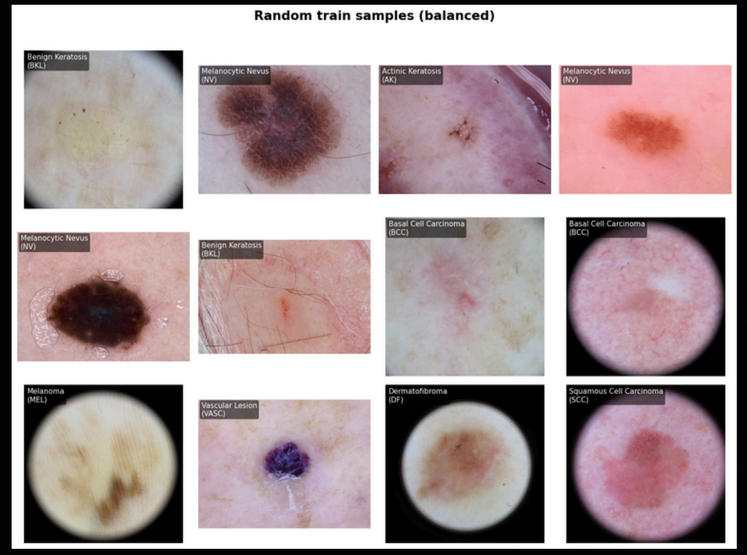
- Healthcare application
- Early detection
- Challenging deep-learning task

Why ISIC?

- Open source
- Real dermoscopic images
- Widely used in literature

What is our goal?

- Design an efficiency-aware CNN
- Improve backbone with lightweight attention
- Trade-off between efficiency and accuracy



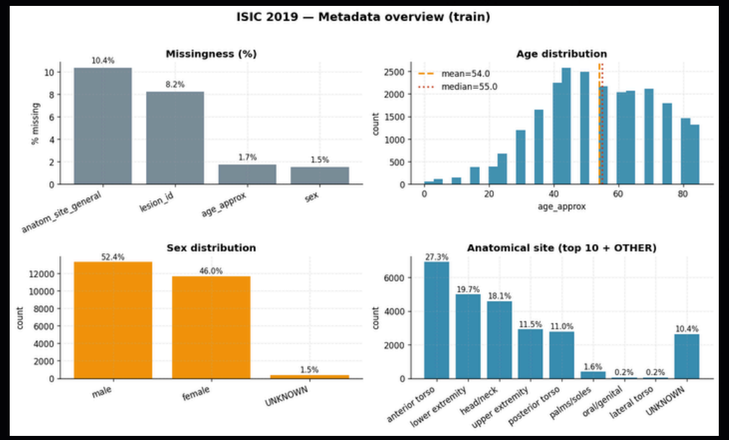
THE DATASET

Two data components
dermoscopic images + tabular metadata

Dermoscopic images
25k train images, 8k test images

Tabular metadata

```
'age_approx', 'anatom_site_general', 'lesion_id', 'sex'
```



9-class classification task

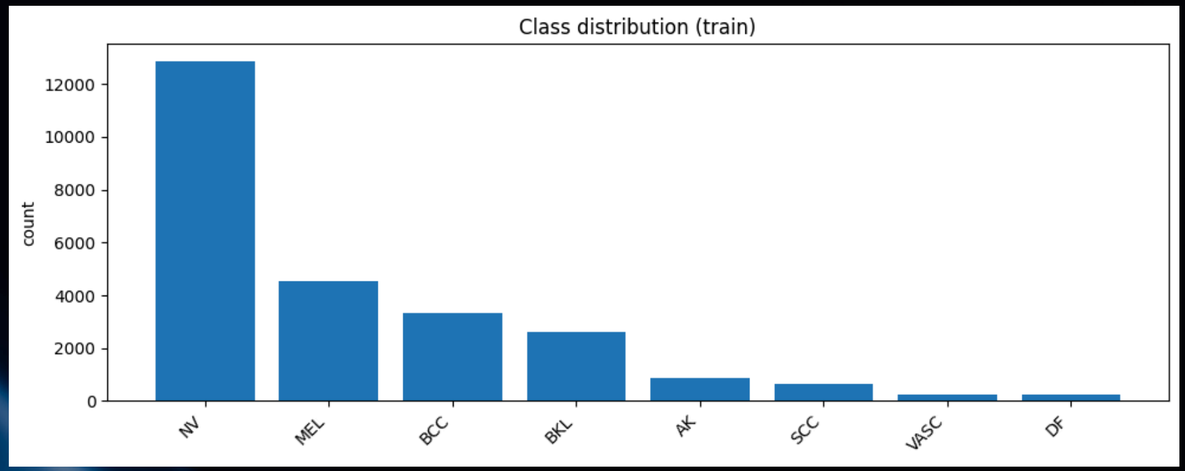
```
'MEL', 'NV', 'BCC', 'AK', 'BKL', 'DF', 'VASC', 'SCC', 'UNK'
```

MAJOR ISSUES — UNK and CLASS IMBALANCE

```
UNK sum TRAIN: 0.0
Training classes (UNK removed): ['MEL', 'NV', 'BCC', 'AK', 'BKL', 'DF', 'VASC', 'SCC']
num_classes: 8
UNK sum TEST: 2047.0
Dropped UNK rows from TEST GT: 2047/8238 removed. Remaining: 6191
```

UNK was removed from test because it was not present in training

Class imbalance must be considered when evaluating results and selecting metrics

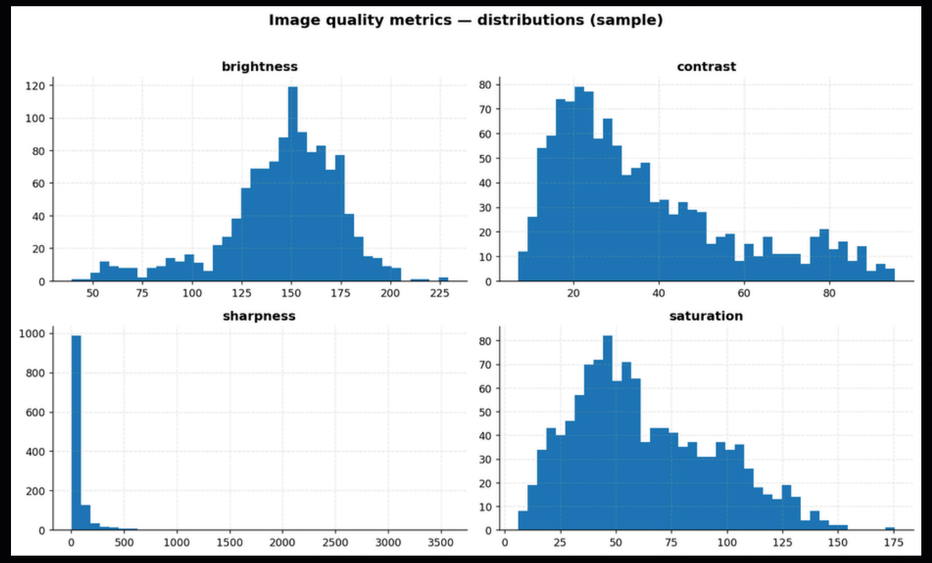


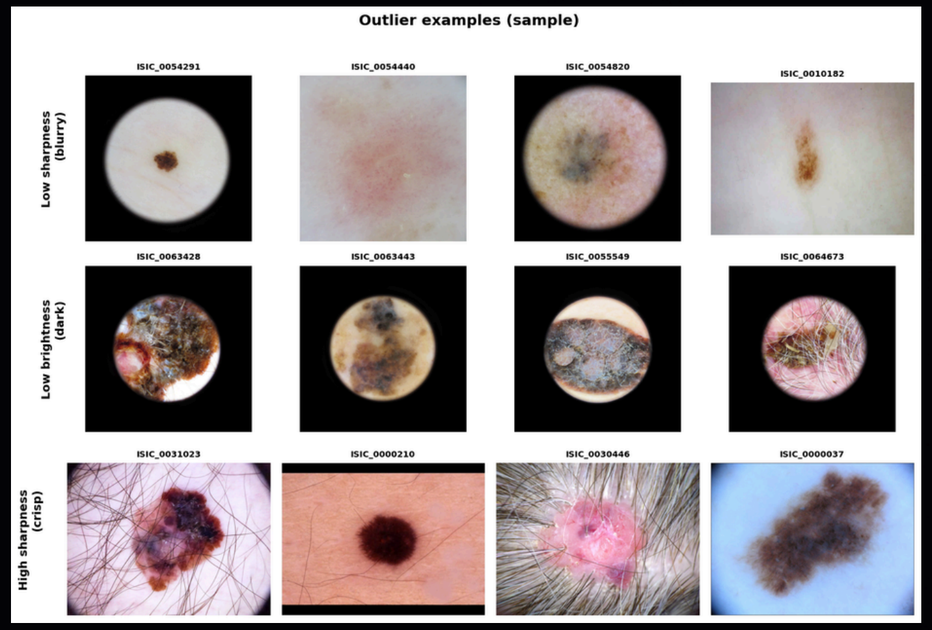
MAJOR ISSUES — IMAGE QUALITY

Input images showed substantial variability due to:

- Non-uniform lighting
- Inconsistent framing
- Variable resolution
- Outlier / low quality samples

To reduce this variability, we normalized the inputs.

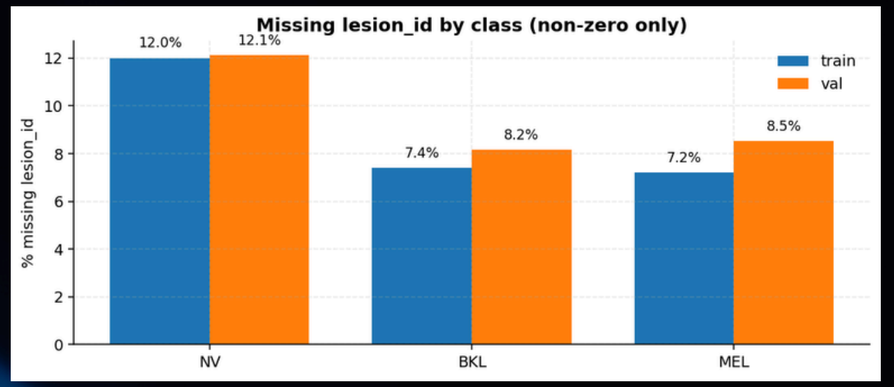


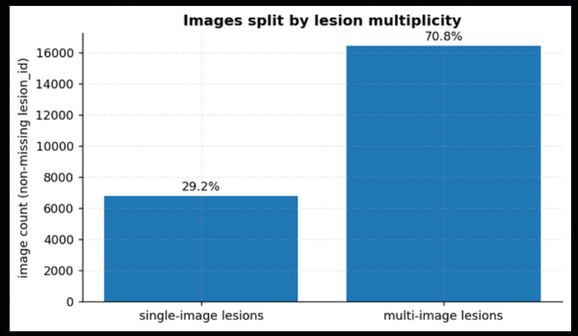


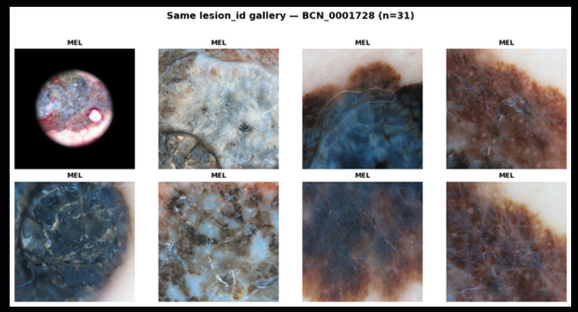
MAJOR ISSUES — LESION ID

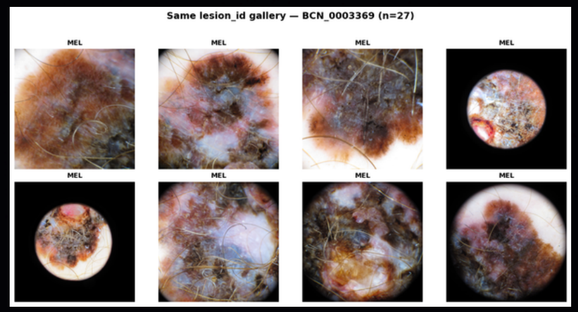
Metadata analysis showed that ***multiple images*** could belong to the same lesion.
This creates a risk of ***data leakage*** between training and validation.
To avoid this, we used a ***lesion-aware*** train/validation split.

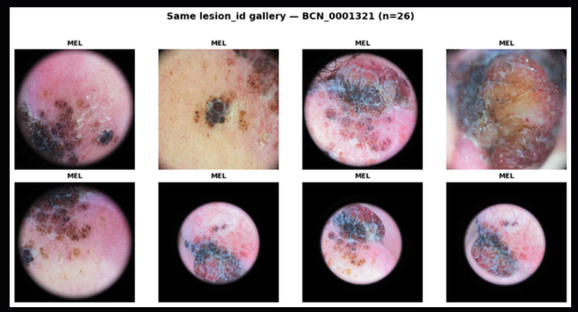
```
=== Lesion-level (lesion_id) summary ===
Total train images: 25331
Missing lesion_id: 2084 (8.2%)
Non-missing images: 23247
Distinct lesions (non-missing): 11847
Max images per lesion: 31
Lesions with >1 image: 5059 (42.7% of lesions)
Images belonging to multi-image lesions: 16459 (70.8% of non-missing images)
```







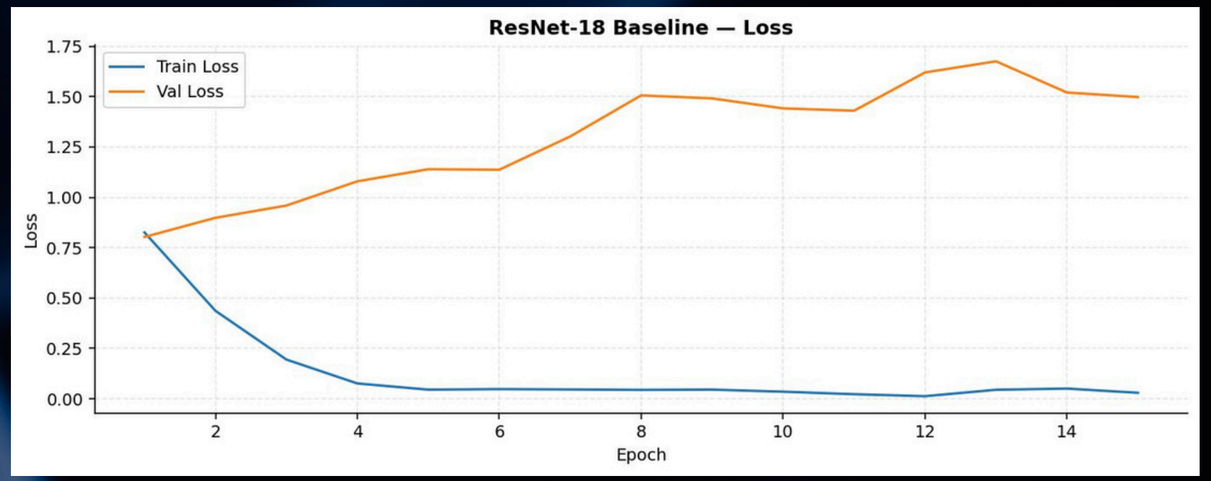


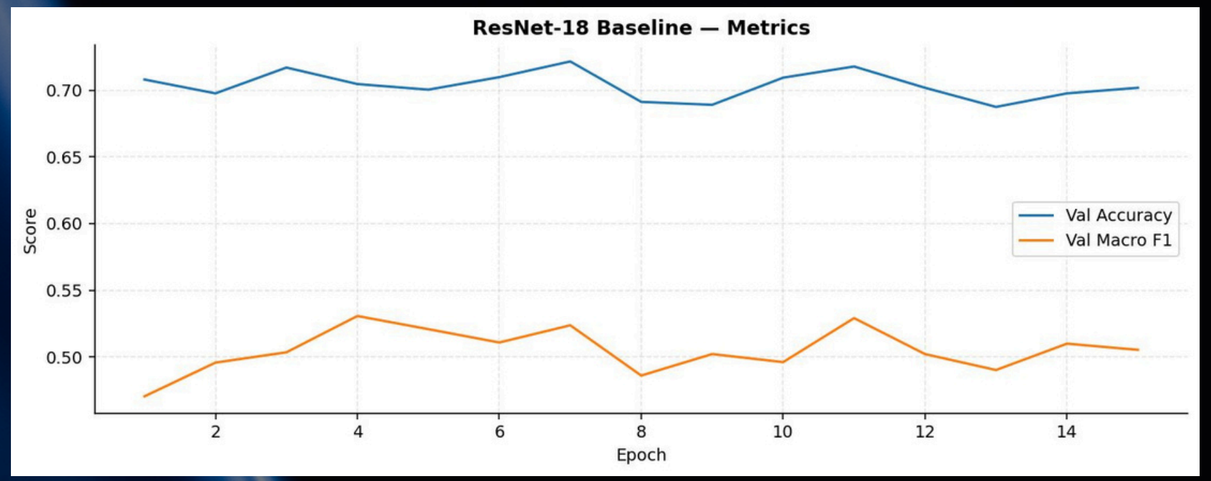


Baseline comparison

- ResNet-18 Vs MobileNetV2 Vs EfficientNet-B0 under the same raw setting
- Show that EfficientNet-B0 achieved the best validation performance
- Explain why ResNet-18 and MobileNetV2 were not selected : lower macro F1 and weaker generalisation / stronger overfitting

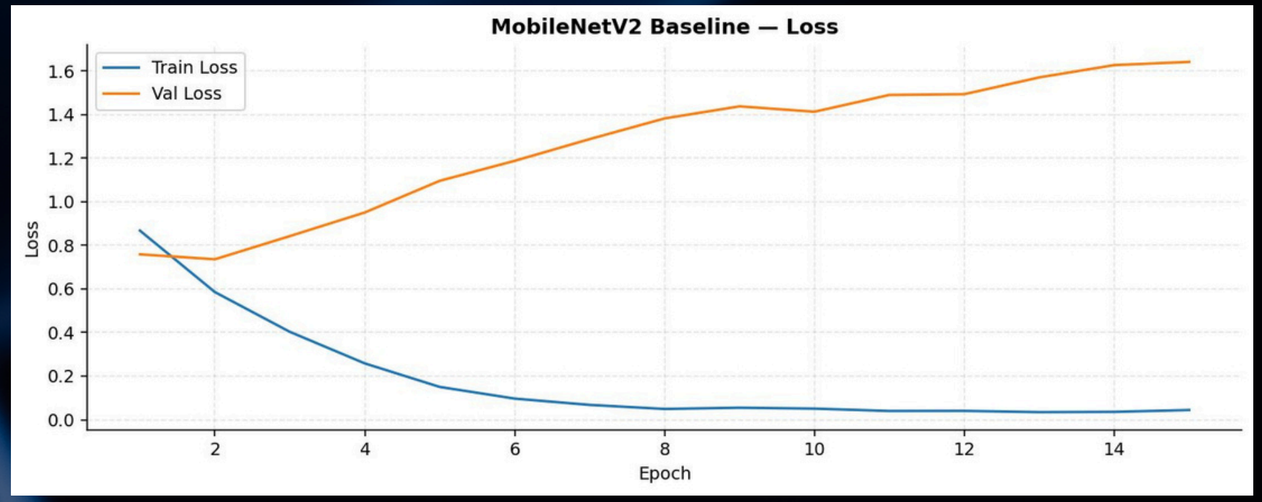
Baseline comparison — ResNet-18



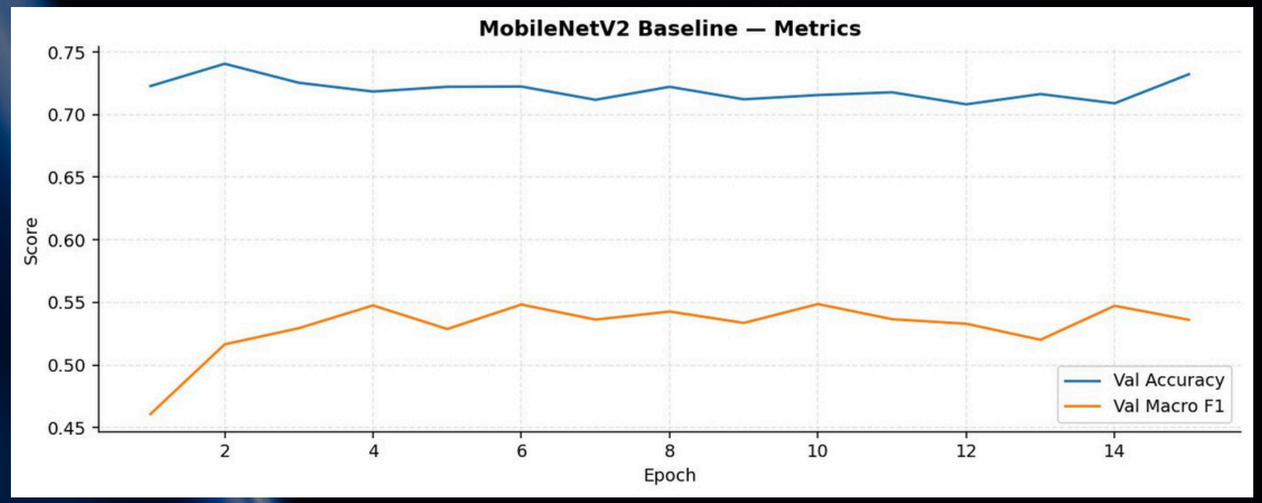


- Here is the performance of using ResNet-18 as the baseline model.
- Diagram on the left shown Validation loss and training loss
- Diagram on the left shown Validation accuracy and Marco F1

Baseline comparison –MobileNetV2

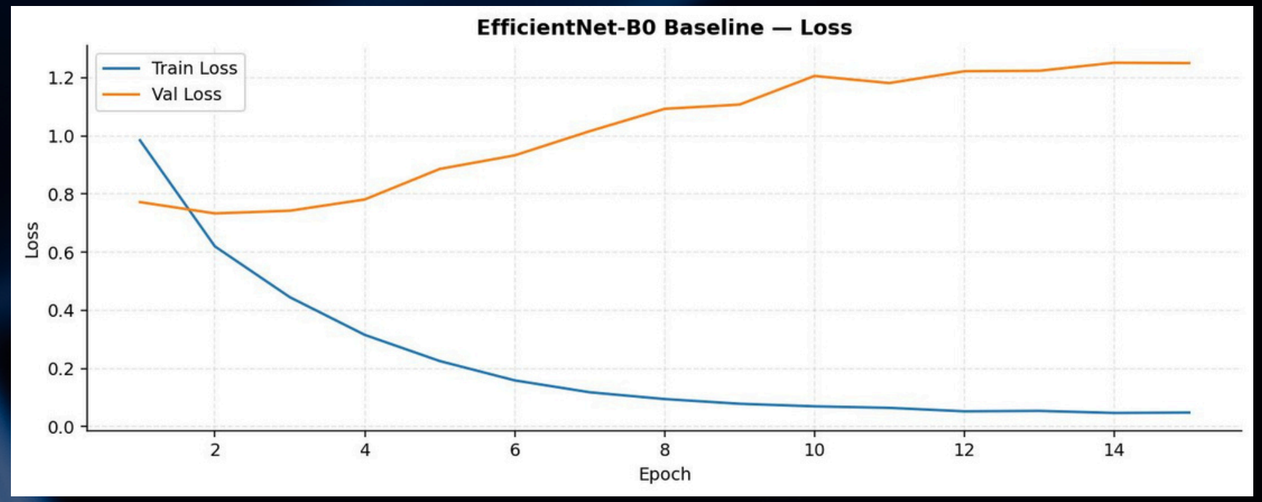


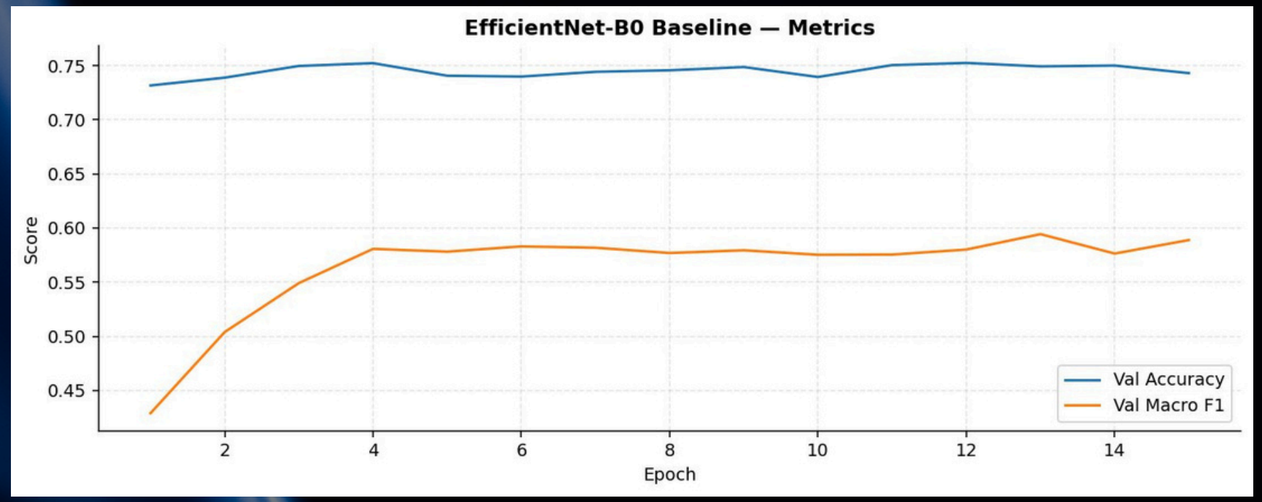
- Here is the performance of using MobileNetV2 as the baseline model.
- Diagram on the left shown Validation loss and training loss



- Diagram on the left shown Validation accuracy and Marco F1

Baseline comparison –EfficientNet-B0





- Here is the performance of using EfficientNet-B0 as the baseline model.
- Diagram on the left shown Validation loss and training loss
- Diagram on the left shown Validation accuracy and Marco F1

Baseline comparison

Model	Final Val Loss	Val Accuracy	Macro F1	Key Observation
ResNet-18	~1.50	~0.70	~0.51	Clear overfitting after epoch 6
MobileNetV2	~1.62	~0.73	~0.55	Highest gap between train/val
EfficientNet-B0	**~1.20**	**~0.75**	**~0.58**	**Best & most stable performance**

- From the previous slide, we can observe that EfficientNet-B0 achieved the best validation performance : higher validation accuracy, higher validation macro F1, and the most stable validation curves across epochs

Improvement strategy

There are four things we have done for better generalization:

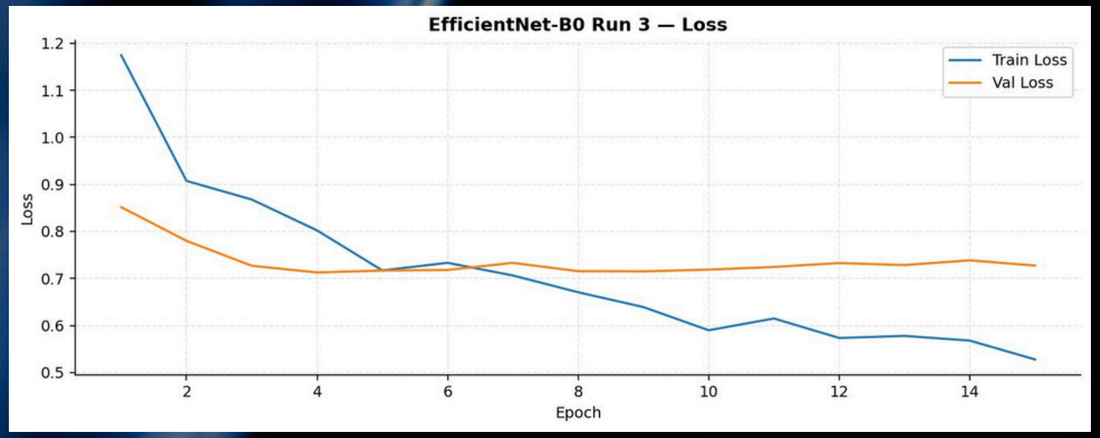
1. Light data augmentation
2. CutMix
3. Cosine annealing scheduler
4. TTA

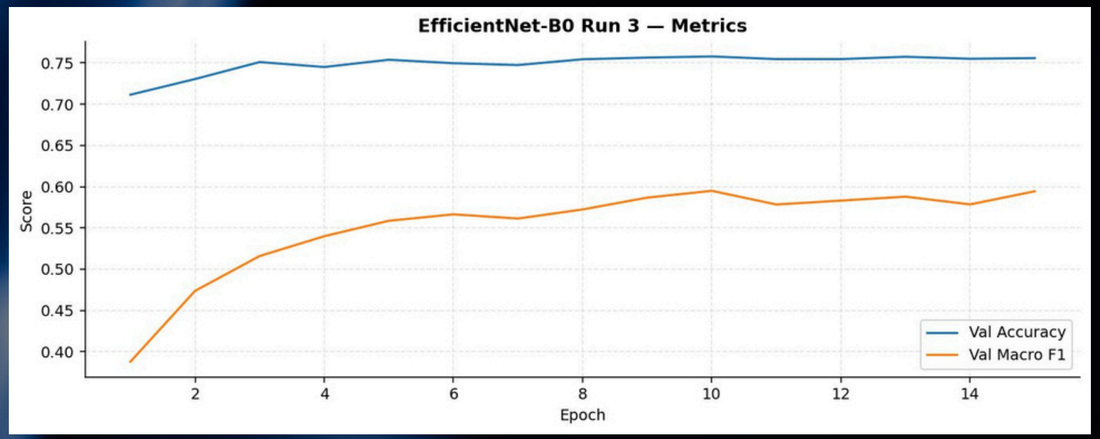
```python
# ------------------------------
# 7. Optimizer + Cosine Scheduler
# ------------------------------
optimizer = optim.Adam(model.parameters(), lr=1e-4)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=15,
    eta_min=1e-6
)
```

code for using Cosine annealing scheduler

```python
# ------------------------------
# 6. CutMix helpers
# ------------------------------
CUTMIX_PROB = 0.5
CUTMIX_ALPHA = 1.0

def rand_bbox(size, lam):
    W = size[2]
    H = size[3]
    cut_rat = np.sqrt(1.0 - lam)
    cut_w = int(W * cut_rat)
    cut_h = int(H * cut_rat)

    cx = np.random.randint(W)
    cy = np.random.randint(H)

    x1 = np.clip(cx - cut_w // 2, 0, W)
    y1 = np.clip(cy - cut_h // 2, 0, H)
    x2 = np.clip(cx + cut_w // 2, 0, W)
    y2 = np.clip(cy + cut_h // 2, 0, H)

    return x1, y1, x2, y2

def apply_cutmix(images, labels, alpha=1.0):
    lam = np.random.beta(alpha, alpha)
    rand_index = torch.randperm(images.size(0), device=images.device)

    target_a = labels
    target_b = labels[rand_index]

    x1, y1, x2, y2 = rand_bbox(images.size(), lam)
    images[:, :, x1:x2, y1:y2] = images[rand_index, :, x1:x2, y1:y2]

    lam = 1 - ((x2 - x1) * (y2 - y1) / (images.size(-1) * images.size(-2)))
    return images, target_a, target_b, lam

# ------------------------------
```

code that implements CutMix

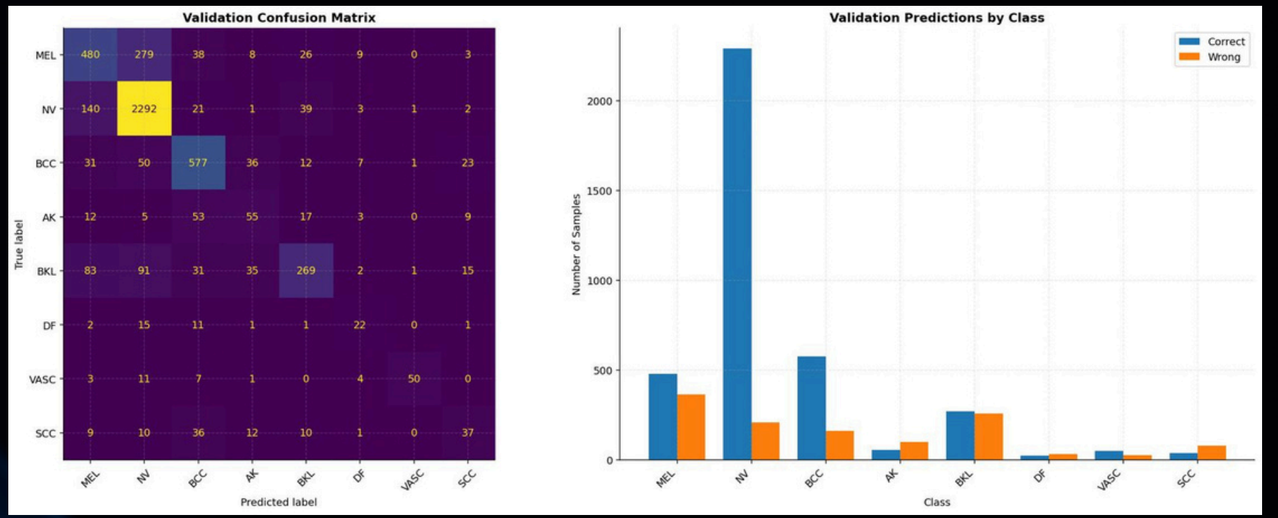





```
TTA Validation Accuracy: 0.7660
TTA Validation Macro F1: 0.6083

Classification Report (TTA):
              precision    recall  f1-score   support

         MEL     0.6633    0.5421    0.5966       843
          NV     0.8287    0.9348    0.8785      2499
         BCC     0.7594    0.7965    0.7775       737
          AK     0.3673    0.3506    0.3588       154
         BKL     0.7383    0.5408    0.6243       527
          DF     0.4400    0.4151    0.4272        53
        VASC     1.0000    0.6579    0.7937        76
         SCC     0.4667    0.3652    0.4098       115

    accuracy                         0.7660      5004
   macro avg     0.6580    0.5754    0.6083      5004
weighted avg     0.7571    0.7660    0.7565      5004
```

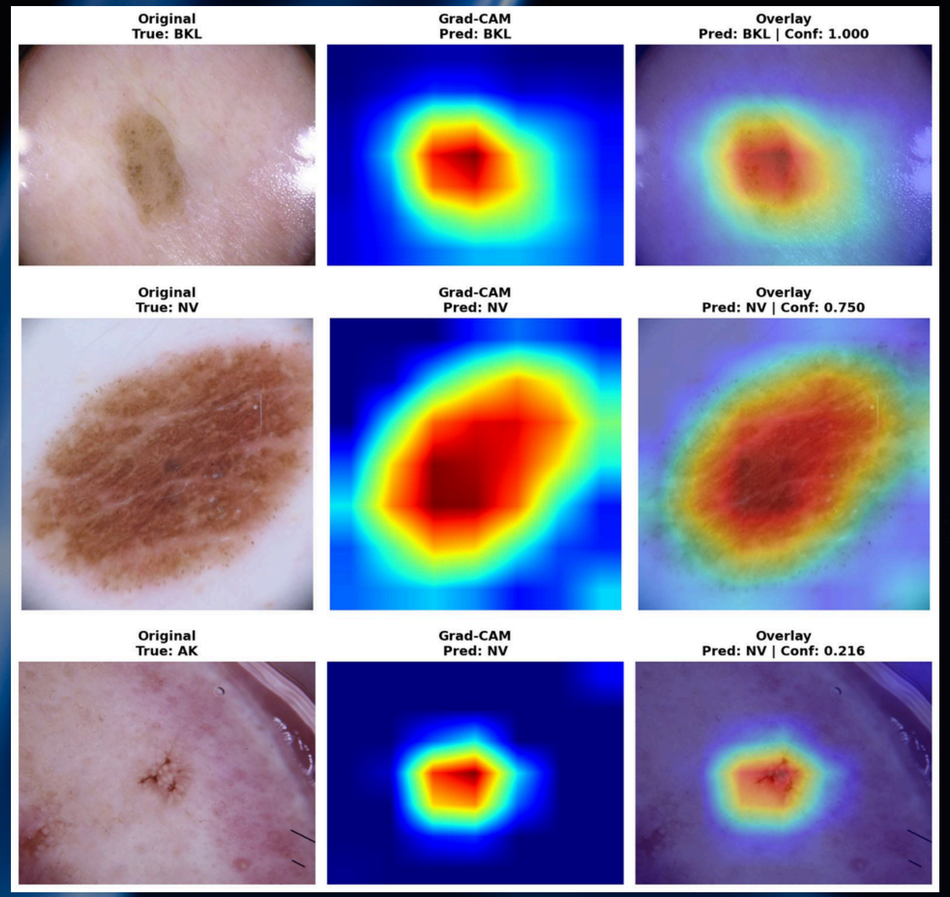
Final Results

- Final validation accuracy: 0.7660
- Macro F1 scores: 0.6083
- Note that balanced accuracy/recall is more important than just considering the accuracy as the dataset is imbalanced
- Weighted recall: 0.7660
- Macro recall: 0.5754
- Both the weighted recall and Marco recall show that performance on the dominant class (i.e. NV) is much better than that on the minority class (e.g. AK, DF, VASC, and SCC)
- The confusion matrix below also shows the performance on dominant and minority classes



Conclusion

- Among the tested baseline architectures, EfficientNet-B0 emerged as the final model due to its superior validation performance and better generalisation compared with ResNet-18 and MobileNetV2.
- In addition, we have performedsome improvement techniques in order to achieve higher accuracy, without increase a lot on the run time cost and hardware requirements
- The study shows that strong classification results can be achieved under a relatively small runtime budget, highlighting the practical value of efficiency-aware model selection for medical image classification.



LIVE DEMO

Final live demo of our model with:

1) user-selected ISIC images

2) user-provided skin lesion image